
Vulnérabilités d'authentification

Comprendre les failles, les tester en laboratoire et mieux les corriger

Gana Faye

Sécurité des applications web | 20 juillet 2026

RISQUE CRITIQUE

À la fin, vous saurez :

- ▶ distinguer authentification, session et autorisation ;
- ▶ expliquer l'origine et l'impact des failles ;
- ▶ reconnaître l'énumération et la force brute ;
- ▶ analyser les protections de mot de passe et de MFA ;
- ▶ auditer les fonctions « rester connecté », réinitialiser et modifier un mot de passe ;
- ▶ proposer une défense en profondeur vérifiable.

Question centrale

Comment l'application prouve-t-elle
que

VOUS ÊTES VOUS ?

Cadre éthique

Les manipulations présentées visent uniquement un laboratoire ou un système pour lequel une autorisation explicite a été obtenue.



Méthode pédagogique

Chaque notion suit le même chemin : **fonctionnement normal** → **erreur de logique** → **observation en laboratoire** → **correction**.

PARTIE 1

Comprendre l'authentification

Qu'est-ce que l'authentification ?

Définition

L'authentification est le processus par lequel un système vérifie l'identité revendiquée par un utilisateur, un service ou un appareil.

- ▶ **Identification** : « Je suis Alice. »
- ▶ **Preuve** : Alice fournit un secret, un jeton ou une caractéristique biométrique.
- ▶ **Session** : le serveur conserve ensuite le résultat de cette vérification.

1. Identité revendiquée



2. Preuve présentée



3. Preuve vérifiée



4. Session créée

Les facteurs d'authentification

Ce que je sais

Mot de passe, phrase secrète, code PIN.

CONNAISSANCE

Ce que je possède

Téléphone, clé matérielle, application d'authentification.

POSSESSION

Ce que je suis

Empreinte digitale, visage ou autre caractéristique biométrique.

INHÉRENCE

Attention au vocabulaire

Deux étapes utilisant le **même facteur** ne constituent pas toujours une véritable MFA. Deux mots de passe restent deux preuves de connaissance.

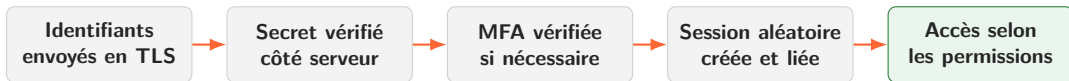
Authentification ou autorisation ?

	Authentification	Autorisation
Question	Qui êtes-vous ?	Que pouvez-vous faire ?
Moment	avant ou pendant la création de session	après identification, à chaque action sensible
Données	mot de passe, code MFA, certificat	rôle, propriétaire, permission, politique
Échec typique	connexion à la place d'une victime	lecture ou modification d'une ressource interdite
Exemple	vérifier le mot de passe d'Alice	vérifier qu'Alice peut lire la facture 42

À retenir

Être connecté ne donne pas automatiquement tous les droits. Une application doit vérifier **l'identité** puis **la permission**.

Une connexion réussie : le parcours complet



Surface d'attaque

Une faiblesse à **n'importe quelle étape** peut annuler la sécurité des autres : comparaison du mot de passe, MFA, création de session, récupération de compte ou changement de secret.

Comment surviennent les failles ?

Défauts techniques

- ▶ mots de passe faibles ou mal stockés ;
- ▶ jetons prévisibles, longs délais d'expiration ;
- ▶ absence de limitation efficace ;
- ▶ cookies mal protégés ;
- ▶ messages ou temps de réponse distincts.

Défauts de logique

- ▶ étape MFA contournable ;
- ▶ utilisateur non lié au défi MFA ;
- ▶ réinitialisation fondée sur une donnée contrôlable ;
- ▶ confiance excessive dans un paramètre client ;
- ▶ incohérence entre plusieurs points d'entrée.

LA LOGIQUE MÉTIER FAIT PARTIE DE LA SÉCURITÉ

Quel est l'impact ?

Utilisateur

Vol de compte, données privées exposées, fraude, usurpation et verrouillage du propriétaire légitime.

Application

Actions au nom de la victime, accès aux fonctions privilégiées, pivot vers d'autres comptes.

Organisation

Incident réglementaire, perte financière, interruption, coût d'enquête et atteinte à la réputation.

Pourquoi le risque est souvent critique

L'attaquant ne casse pas seulement une page : il hérite de la **confiance**, des **données** et parfois des **privilèges** attachés au compte.

- 1 **Compte client** : consultation d'informations personnelles, commandes frauduleuses, modification de l'adresse ou du moyen de contact.
- 2 **Compte employé** : accès aux documents internes, messagerie, outils métiers et données d'autres utilisateurs.
- 3 **Compte administrateur** : création de comptes, changement de configuration, extraction massive ou compromission complète.

Compte compromis

Données et actions

Escalade et persistance

PARTIE 2

Vulnérabilités de la connexion par mot de passe

```
POST /login HTTP/1.1
Host: lab.example
Content-Type: application/x-www-form-urlencoded

username=alice&password=Demo-2026!
```

Ce que le serveur doit faire

Normaliser prudemment l'identifiant, retrouver le compte, vérifier le hachage du secret, appliquer la limitation, journaliser, puis créer une session neuve.

Indices à observer

- ▶ code HTTP ;
- ▶ longueur du corps ;
- ▶ texte du message ;
- ▶ redirection ;
- ▶ cookie créé ;
- ▶ temps de réponse ;
- ▶ comportement après répétition.

Définition

La force brute consiste à tester systématiquement plusieurs valeurs jusqu'à trouver une combinaison acceptée.

Variantes fréquentes

- ▶ plusieurs mots de passe pour un compte ;
- ▶ plusieurs comptes avec un mot de passe courant ;
- ▶ listes issues de fuites anciennes ;
- ▶ codes courts ou jetons insuffisamment aléatoires.

Facteurs de succès

- ▶ secrets faibles ou réutilisés ;
- ▶ espace de recherche réduit ;
- ▶ réponses révélatrices ;
- ▶ absence de délai ou de limitation ;
- ▶ protection contournable.

AUTOMATISATION + SIGNAL FIABLE = RISQUE



La faille exacte

Le serveur fournit un signal fiable pour chaque essai et autorise assez de tentatives pour explorer une partie utile de l'espace des secrets.

Ce qui casse la chaîne

Limitation adaptative, mots de passe non compromis, MFA, détection globale et réponses qui ne révèlent aucun détail supplémentaire.

Objectif

Déterminer quels identifiants correspondent à de vrais comptes avant de tester les mots de passe.

- ▶ adresses e-mail au format prévisible ;
- ▶ auteurs visibles sur le site ;
- ▶ identifiants présents dans des profils ou documents ;
- ▶ différences produites par la page de connexion.

Erreur de conception

Le serveur traite « utilisateur absent » et « mot de passe faux » de façon observablement différente.

Défense

Message générique, chemin de traitement uniforme et limitation tenant compte du compte, de l'origine et du risque.

Pourquoi certains secrets tombent vite

- ▶ mots du dictionnaire ;
- ▶ saison + année ;
- ▶ prénom + chiffre ;
- ▶ substitutions prévisibles : a → @ ;
- ▶ réutilisation sur plusieurs services.

Politique plus robuste

- ▶ favoriser les phrases longues ;
- ▶ bloquer les mots de passe compromis ;
- ▶ autoriser les gestionnaires de mots de passe ;
- ▶ éviter les règles de composition artificielles ;
- ▶ ajouter une MFA résistante au phishing.

Point important

Un mot de passe complexe ne compense pas une logique de connexion qui divulgue les comptes ou accepte un nombre illimité d'essais.

Énumération des noms d'utilisateur

Canal	Signal possible	Bonne pratique
Message	« utilisateur inconnu » contre « mot de passe faux »	réponse générique identique
Code HTTP	200 contre 302 ou 403	comportement cohérent
Longueur	page contenant une phrase supplémentaire	gabarit uniforme
Temps	hachage effectué seulement pour un compte réel	vérification factice de coût comparable
Verrouillage	seul un compte existant se bloque	mécanisme ne servant pas d'oracle

Un « oracle »

Toute différence répétable permettant de répondre à une question secrète : « ce compte existe-t-il ? », « ce code est-il correct ? ».

- 1 Capturer une requête de connexion du laboratoire avec Burp Proxy.
- 2 L'envoyer dans Intruder et marquer uniquement le champ `username`.
- 3 Utiliser une petite liste fournie par le laboratoire.
- 4 Comparer **statut**, **longueur** et **message**.
- 5 Confirmer manuellement l'anomalie avant toute conclusion.

Résultat attendu

Une réponse se distingue nettement : elle révèle probablement un identifiant valide.

Ne pas confondre

Une différence isolée peut venir du réseau, d'un jeton CSRF ou d'un contenu dynamique. La répétabilité est essentielle.

- 1 Dans **Proxy** > **HTTP history**, sélectionner la requête POST /login.
- 2 Choisir **Send to Intruder**, puis conserver une seule position :

```
username=$alice&password=invalid-demo
```

- 3 Charger les identifiants fournis par le laboratoire.
- 4 Garder le même mot de passe volontairement faux pour isoler la variable username.

Pourquoi une seule position ?

Si le nom et le mot de passe changent ensemble, on ne sait plus quelle donnée explique la différence observée.

Contrôle expérimental

Une bonne expérience ne modifie qu'une variable à la fois et conserve une requête de référence.

Candidat	Statut	Longueur	Réponse observée
user-test-1	200	3210	Invalid username
user-test-2	200	3210	Invalid username
carlos	200	3251	Incorrect password
user-test-3	200	3210	Invalid username

Interprétation

carlos produit une réponse différente : le serveur a probablement trouvé le compte avant d'échouer sur le mot de passe.

Validation

Répéter manuellement avec Repeater. Si la différence est stable, l'oracle d'énumération est confirmé.

```
Echec A : Invalid username or password.  
Echec B : Invalid username or password
```

Une ponctuation, une espace, une casse ou une balise invisible peut suffire à créer un oracle.

Technique d'analyse

Rechercher une chaîne stable, trier les réponses, puis utiliser un comparateur de contenu plutôt que la seule taille brute.

Pièges

- ▶ contenu personnalisé ;
- ▶ horodatage ;
- ▶ identifiant de requête ;
- ▶ ordre variable des attributs ;
- ▶ compression.

COMPARER LE SENS

Hypothèse de laboratoire

Pour un utilisateur réel, le serveur calcule un hachage coûteux ; pour un utilisateur absent, il répond immédiatement.

- 1 mesurer plusieurs essais de référence ;
- 2 répéter chaque candidat, jamais une seule fois ;
- 3 comparer médiane et dispersion ;
- 4 confirmer depuis les mêmes conditions réseau.

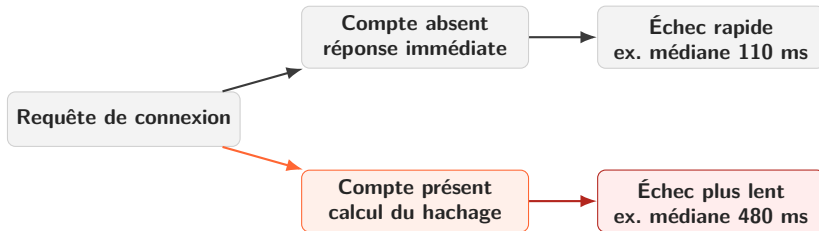
Pourquoi c'est délicat

Latence, charge serveur, cache et limitation créent du bruit. Une différence doit être **stable et statistiquement crédible**.

Correction

Effectuer une vérification de coût comparable même lorsque le compte n'existe pas.

TP temporel : pourquoi le délai révèle le compte



Procédure fiable

Envoyer plusieurs séries, alterner les candidats, ignorer les premières mesures, puis comparer les **médianes**. Un seul essai lent ne prouve rien.

Cause racine

Les deux branches exécutent un travail différent. Le serveur devrait effectuer une vérification factice de coût équivalent pour un utilisateur absent.

Protections fragiles

- ▶ blocage fondé uniquement sur l'adresse IP ;
- ▶ compteur réinitialisé après une connexion valide ;
- ▶ limitation appliquée à une route mais pas aux autres ;
- ▶ confiance dans un en-tête contrôlable ;
- ▶ seuil identique pour tous les risques.

Défense en profondeur

- ▶ compte + IP + appareil + contexte ;
- ▶ délai progressif ;
- ▶ détection des mots de passe pulvérisés ;
- ▶ MFA ou défi adaptatif ;
- ▶ alertes et télémétrie centralisée.

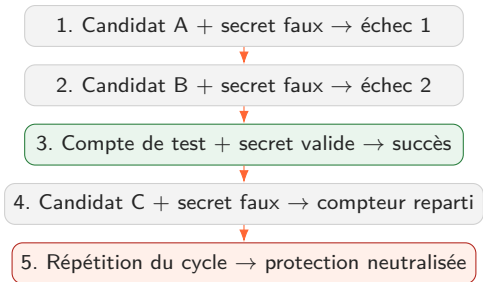
Logique vulnérable typique

Après plusieurs échecs, l'IP est temporairement bloquée ; mais une connexion valide remet le compteur à zéro sans traiter le comportement global.



Leçon

Une protection ne doit pas être évaluée seulement dans le « chemin normal » : il faut tester ses transitions, remises à zéro et interactions avec les connexions réussies.



Ce que teste le TP

La remise à zéro du compteur, pas seulement le seuil de blocage.

Pourquoi c'est une faille

Une connexion valide intercalée permet de continuer les essais depuis la même origine. La protection raisonne requête par requête au lieu d'identifier une campagne.

Résultat vulnérable

Les essais faux sont acceptés indéfiniment tant qu'un succès connu apparaît avant le seuil. La limite IP existe, mais son état est manipulable.

Preuve à conserver

Suite ordonnée des requêtes, codes HTTP, messages, horodatages et absence de blocage au-delà du seuil théorique.

Correction précise

- ▶ ne pas effacer tout l'historique après un succès ;
- ▶ suivre compte, origine et campagne globale ;
- ▶ appliquer un délai progressif ;
- ▶ détecter la répétition du motif échec/succès ;
- ▶ imposer un défi renforcé selon le risque.

Avantage

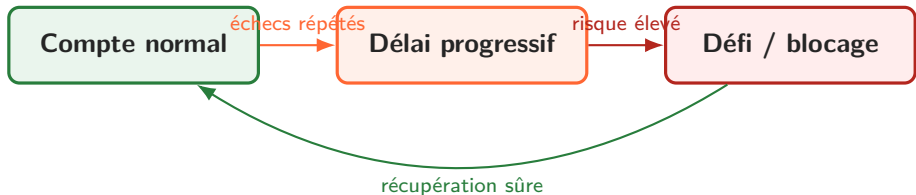
Réduit le nombre de tentatives possibles contre un compte ciblé et ralentit l'attaque.

Risques

- ▶ déni de service contre les utilisateurs ;
- ▶ révélation de l'existence du compte ;
- ▶ support saturé ;
- ▶ contournement via une autre route.

Approche préférable

- ▶ délais progressifs plutôt qu'un verrou permanent ;
- ▶ notification au propriétaire ;
- ▶ récupération sûre ;
- ▶ analyse du risque ;
- ▶ protection cohérente sur toutes les interfaces.



Questions d'audit

Qu'est-ce qui incrémente le compteur ? Qu'est-ce qui le remet à zéro ? Combien de temps dure l'état ? Le comportement est-il identique sur le web, l'API et l'application mobile ?

- 1 Choisir des identifiants factices du laboratoire.
- 2 Répéter exactement le même nombre d'échecs pour chacun.
- 3 Observer message, code, délai et comportement ultérieur.
- 4 Identifier si un seul candidat entre dans l'état « verrouillé ».
- 5 Confirmer sans prolonger inutilement le blocage.

Oracle de verrouillage

Si seuls les comptes réels peuvent être verrouillés, l'état du mécanisme révèle leur existence.

Correction

Réponses externes uniformes, détection interne et contrôles anti-abus qui ne créent pas de signal exploitable.

TP verrouillage : matrice d'observation

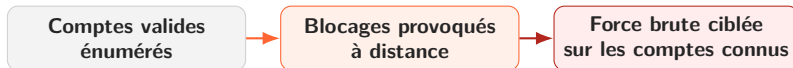
Identifiant	Essais faux	État final	Interprétation
inconnu-1	5	réponse normale	aucune donnée de compte trouvée
inconnu-2	5	réponse normale	aucune donnée de compte trouvée
carlos	5	compte temporaire- ment bloqué	le compte existe probablement
inconnu-3	5	réponse normale	aucune donnée de compte trouvée

Vulnérabilité précise

L'existence du compte influence un changement d'état visible. Le verrouillage devient un oracle d'énumération.

Bonne preuve

Même nombre d'essais, même mot de passe faux, même rythme et même origine pour tous les candidats.



Correction équilibrée

Conserver une réponse externe générique, utiliser des délais progressifs, notifier le propriétaire, prévoir une récupération sûre et détecter les abus sans exposer l'état interne du compte.

À éviter

Un verrouillage permanent et immédiat permettrait à n'importe qui de rendre indisponibles des comptes connus.

Dimension	Seule	Faiblesse	Rôle combiné
Adresse IP	bloque une source	IP partagée ou changeante	détecter une origine agressive
Compte	protège une cible	permet le déni de service	détecter un ciblage précis
Appareil/session Réseau global	suit un client voit la campagne	empreinte imparfaite faux positifs possibles	enrichir le contexte détecter la pulvérisation

Bon modèle

Une décision adaptative combine plusieurs signaux, ajoute un délai progressif et déclenche une vérification renforcée sans révéler la cause exacte.

```
GET /admin HTTP/1.1  
Host: lab.example  
Authorization: Basic YWxpY2U6ZGVtby1wYXNz
```

Fonctionnement

Le navigateur envoie nom:mot-de-passe encodé en Base64 dans un en-tête à chaque requête concernée.

Base64 n'est pas du chiffrement

Toute personne qui récupère la valeur peut la décoder. TLS est indispensable, mais ne corrige pas les secrets faibles ni la force brute.

- ▶ déconnexion peu élégante ;
- ▶ identifiants longtemps réutilisés ;
- ▶ protections parfois moins riches.

À vérifier

- ▶ HTTPS obligatoire et HSTS ;
- ▶ pas d'identifiants dans une URL ;
- ▶ limitation des tentatives ;
- ▶ permissions minimales ;
- ▶ secrets distincts par service ;
- ▶ absence de journalisation de l'en-tête.

Quand l'éviter

Pour une application utilisateur moderne nécessitant MFA, révocation précise, déconnexion globale, gestion de session ou analyse de risque avancée.

Principe

Un mécanisme simple peut être approprié dans un périmètre restreint, mais il doit être entouré de contrôles cohérents.

PARTIE 3

Vulnérabilités de l'authentification multifactorielle

Pourquoi ajouter un second facteur ?

But

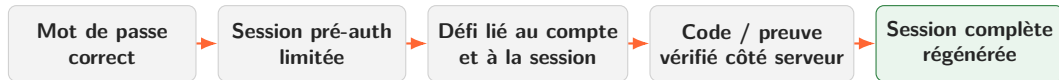
Empêcher qu'un mot de passe volé suffise à ouvrir une session.

- ▶ code TOTP produit par une application ;
- ▶ notification ou validation sur un appareil ;
- ▶ clé de sécurité matérielle ;
- ▶ passkey fondée sur la cryptographie asymétrique.

La MFA n'est pas magique

Elle échoue si l'étape peut être sautée, si le défi n'est pas lié au bon utilisateur, si le code est bruteforçable ou si la récupération est plus faible.

CHAÎNE DE SÉCURITÉ COHÉRENTE



Invariant de sécurité

Aucune ressource nécessitant une authentification complète ne doit être accessible tant que l'état MFA n'a pas été validé côté serveur.

Contournement de flux

- ▶ accès direct à une page après le mot de passe ;
- ▶ paramètre client `mfa=true` ;
- ▶ API mobile moins protégée ;
- ▶ session complète créée trop tôt.

Vérification faible

- ▶ code non lié au compte ;
- ▶ code réutilisable ou trop longtemps valide ;
- ▶ essais illimités ;
- ▶ canal de récupération faible ;
- ▶ absence d'alerte ou de révocation.

Jetons d'authentification à deux facteurs

Méthode	Atout	Limite	Usage prudent
SMS	large disponibilité	interception, SIM swap, phishing	mieux que mot de passe seul
E-mail	simple	dépend de la sécurité de la boîte	récupération encadrée
TOTP	hors ligne, court	phishable, secret partagé	limitation stricte
Push	confortable	fatigue de validation	contexte + nombre matching
Clé / passkey	résistante au phishing	déploiement à prévoir	privilégiée pour le risque élevé

Propriété essentielle

Le serveur doit vérifier la preuve, son expiration, son usage unique et son lien avec **le compte et la transaction en cours**.

Scénario vulnérable

Après le mot de passe, l'application redirige vers /login2. Mais la session possède déjà les mêmes droits qu'après la MFA.

```
POST /login    -> 302 /login2
GET  /my-account -> 200 OK    # anomalie
```

Erreur conceptuelle

La redirection impose une étape à l'interface, pas au serveur. Un attaquant peut demander directement la ressource finale.

Correction

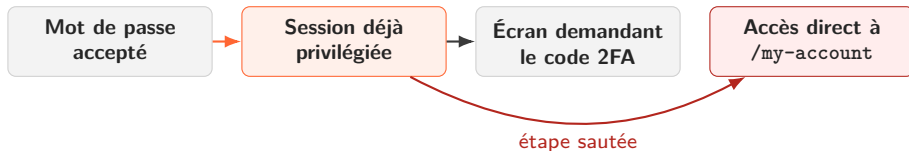
État pré-authentifié distinct, contrôle centralisé sur chaque route et session régénérée après la MFA.

- ❶ Se connecter avec le compte de test et observer la transition vers la page du code.
- ❷ Examiner le cookie et les requêtes produites avant la validation MFA.
- ❸ Demander une page normalement réservée au compte authentifié.
- ❹ Vérifier si le serveur contrôle réellement l'état MFA.
- ❺ Documenter l'invariant violé et la réponse attendue : refus ou redirection contrôlée.

Preuve minimale

Il suffit de démontrer l'accès à une ressource bénigne du laboratoire. Ne pas multiplier les actions une fois la faille confirmée.

TP 2FA simple : visualiser le défaut d'état



Vulnérabilité précise

La 2FA n'est imposée que par la navigation de l'interface. Le contrôle d'accès ne vérifie pas l'état `mfa_verified` sur la ressource finale.

Résultat attendu du TP

Avant le code, `/my-account` devrait répondre par un refus. Une réponse 200 OK contenant les données du compte confirme le contournement.

- 1 Intercepter la réponse à POST /login et noter le cookie créé.
- 2 Laisser le navigateur atteindre /login2, sans saisir le code.
- 3 Dans Repeater, réutiliser ce cookie pour demander GET /my-account.
- 4 Comparer le résultat avant et après validation 2FA.

Cas sûr

Session pré-authentifiée limitée ; la page finale renvoie 401, 403 ou une redirection vers le défi.

Cas vulnérable

Le même cookie donne déjà accès au compte. Le code 2FA devient décoratif.

```
POST /login2 HTTP/1.1
Cookie: verify=carlos

mfa-code=123456
```

Question critique

Qui choisit l'utilisateur à vérifier : le serveur à partir d'une session sûre, ou le client à partir d'un cookie modifiable ?

Faille

Si le serveur fait confiance à `verify`, un défi initié par une personne peut être réorienté vers un autre compte.

Correction

Lier le défi à un identifiant opaque stocké côté serveur, à la session pré-authentifiée et au compte déjà validé par mot de passe.

- 1 Cartographier les requêtes : génération du code, affichage du formulaire, vérification, création de session.
- 2 Identifier chaque donnée qui désigne le compte : cookie, paramètre, chemin, jeton ou objet JSON.
- 3 Modifier une seule donnée à la fois dans le laboratoire.
- 4 Vérifier si le serveur mélange le défi d'une session avec l'identité d'une autre.
- 5 Contrôler la limitation des essais et l'expiration du défi.

Conclusion attendue

La preuve MFA doit être validée dans un contexte serveur indivisible : **compte + session + défi + expiration + nombre d'essais**.



Vulnérabilité précise

L'identité concernée par le code est choisie par une donnée contrôlée par le client, au lieu d'être fixée dans l'état serveur créé après le mot de passe.

Invariant correct

challenge_id doit désigner côté serveur un unique tuple : compte, session pré-authentifiée, expiration et compteur d'essais.

- 1 Se connecter au compte de test et provoquer la génération de son propre défi.
- 2 Repérer le cookie ou paramètre qui désigne l'utilisateur à vérifier.
- 3 Dans le laboratoire, remplacer uniquement cette valeur par l'identifiant cible fourni.
- 4 Demander un nouveau code, puis observer si le défi est désormais associé à la cible.
- 5 Tester quelques codes factices pour vérifier si la limitation porte sur la bonne combinaison compte/défi/session.

Preuve de faille

La réponse confirme ou traite un défi pour un compte qui n'a pas validé son mot de passe dans cette session.

Correction

Supprimer l'identité contrôlable et retrouver le compte exclusivement depuis l'état pré-authentifié stocké côté serveur.

Pourquoi les codes courts sont sensibles

Un code à six chiffres possède un espace fini. Sans limitation, expiration et invalidation, l'automatisation devient réaliste.

- ▶ code valide trop longtemps ;
- ▶ compteur par IP seulement ;
- ▶ nouveau défi sans invalider l'ancien ;
- ▶ réponses révélant « expiré » ou « faux ».

Contrôles nécessaires

- ▶ très peu d'essais par défi et par compte ;
- ▶ expiration courte ;
- ▶ usage unique ;
- ▶ révocation après succès ou nouveau défi ;
- ▶ alerte et réauthentification ;
- ▶ préférence pour une MFA anti-phishing.

- 1 Capturer la requête de vérification du code avec un compte de test.
- 2 Envoyer la requête dans Intruder et marquer uniquement `mfa-code`.
- 3 Utiliser la plage réduite prévue par le laboratoire, avec un débit prudent.
- 4 Trier les résultats par statut, longueur, redirection et cookie.
- 5 Confirmer le candidat distinct dans Repeater avant expiration.

Ce que démontre le TP

Le code possède un espace court et le serveur n'impose pas assez peu d'essais par défi.

Réponse sûre

Après quelques erreurs : défi invalidé, nouvelle authentification requise, délai et alerte.

Code 2FA : distinguer les réponses

Code testé	Statut	Redirection	Interprétation
000001	200	aucune	code refusé, formulaire réaffiché
000002	200	aucune	code refusé, formulaire réaffiché
000003	302	/my-account	preuve acceptée, session complète créée

Signal de succès

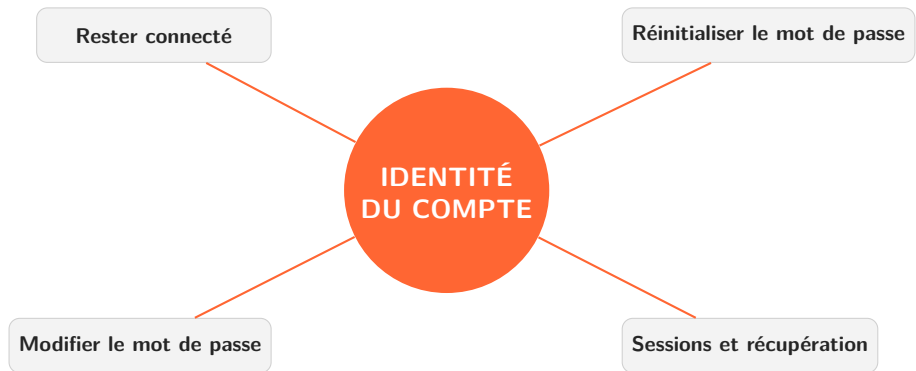
Ne pas se fier uniquement au texte. La redirection et la création d'un nouveau cookie de session sont souvent des signaux plus robustes.

Correction de conception

Compteur atomique par défi et par compte, expiration courte, invalidation après le seuil, usage unique et journalisation de l'événement.

PARTIE 4

Autres mécanismes d'authentification



Principe

Un mécanisme secondaire ne doit jamais permettre de contourner les garanties exigées par le mécanisme principal.

Fonction normale

Un cookie persistant permet de recréer une session après la fermeture du navigateur, sans redemander immédiatement le mot de passe.

Le jeton doit être :

- ▶ aléatoire et non dérivable du mot de passe ;
- ▶ stocké haché côté serveur ;
- ▶ limité dans le temps et révocable ;
- ▶ renouvelé après usage ;
- ▶ protégé par Secure, HttpOnly, SameSite.

Conceptions dangereuses

`base64(username:password)`,
hachage sans secret serveur, jeton
permanent, absence de révocation ou
cookie réutilisable après changement
de mot de passe.

Mauvais modèle

```
remember = base64(username + ":" + md5(
    password))
```

Prévisible, lié au secret utilisateur et souvent vérifiable hors ligne.

Bon modèle

```
selector = random(128 bits)
validator = random(256 bits)
db = selector + hash(validator) + expiry
```

Le cookie contient le sélecteur et le validateur ; le serveur compare le hachage et renouvelle le jeton.

- 1 Activer l'option dans le laboratoire et comparer les cookies avant/après.
- 2 Décoder uniquement les encodages évidents afin d'identifier la structure.
- 3 Modifier séparément l'identifiant, le jeton et les séparateurs.
- 4 Tester expiration, déconnexion, changement de mot de passe et révocation.
- 5 Déterminer si le serveur conserve un état permettant d'invalider le jeton.

Question de fond

Le cookie est-il une référence aléatoire contrôlée par le serveur, ou une preuve fabriquée à partir de données prévisibles ?

TP cookie persistant : décoder le mécanisme



Vulnérabilité précise

Le cookie n'est pas un jeton aléatoire géré par le serveur : il est calculé à partir de l'identité et du mot de passe. Sa structure aide à deviner ou vérifier le secret.

Manipulations utiles

Comparer deux connexions, décoder la valeur, changer uniquement le nom, puis vérifier si déconnexion et changement de mot de passe invalident l'ancien cookie.

TP cookie persistant : tableau de validation

Test	Résultat sûr	Résultat vulnérable
Cookie après déconnexion	refusé	toujours accepté
Cookie après changement du mot de passe	refusé	toujours accepté
Modification de l'identifiant	signature invalide	accès à un autre compte
Réutilisation du même jeton	rotation ou détection	validité illimitée
Décodage de la valeur	données opaques	identité + hachage prévisible

Correction précise

Générer un validateur aléatoire, stocker seulement son hachage côté serveur, limiter sa durée, effectuer une rotation à chaque usage et révoquer toute la famille de jetons en cas de réutilisation suspecte.

En ligne

Chaque essai interroge le serveur : la limitation, les journaux et les alertes peuvent intervenir.

Hors ligne

Après récupération d'un hachage vérifiable, les essais se font sans contacter le serveur. La défense dépend alors du coût du hachage et de la qualité du secret.

Stockage recommandé

- ▶ fonction dédiée aux mots de passe ;
- ▶ sel unique par mot de passe ;
- ▶ paramètres de coût adaptés ;
- ▶ éventuel *pepper* protégé séparément ;
- ▶ migration progressive des anciens hachages.



Différence essentielle

Le serveur ne voit aucun essai : ni limitation, ni verrouillage, ni alerte de connexion ne peuvent intervenir pendant la comparaison locale.

But pédagogique du TP

Montrer pourquoi un hachage rapide ou un cookie contenant un dérivé direct du mot de passe transforme une fuite limitée en attaque répétable hors ligne.

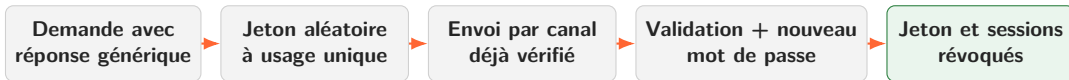
- 1 Extraire uniquement la valeur fournie par le laboratoire.
- 2 Identifier l'encodage et l'algorithme indiqué par l'exercice.
- 3 Calculer localement le hachage d'une courte liste de démonstration.
- 4 Comparer les valeurs et confirmer la correspondance sur le compte de test.
- 5 Révoquer le jeton ou restaurer le secret après la preuve.

Ce qui ralentit réellement

Argon2id, scrypt ou bcrypt correctement paramétré, sel unique, mot de passe long et éventuel pepper protégé séparément.

Ce qui ne suffit pas

Base64, SHA rapide seul ou « algorithme secret » sans sel ni coût adapté.



Le lien de récupération est une clé temporaire

Toute personne qui le possède peut souvent choisir un nouveau mot de passe. Il doit donc être secret, court, unique, lié au compte et invalidé immédiatement après usage.

À ne pas faire

Envoyer le mot de passe actuel prouve qu'il est récupérable en clair ou par un mécanisme réversible. Envoyer un nouveau mot de passe permanent expose aussi un secret durable.

À faire

Envoyer un lien contenant un jeton temporaire, puis laisser l'utilisateur choisir un nouveau secret sur le site HTTPS légitime.

- ▶ ne jamais inclure le secret dans les journaux ;
- ▶ notifier le changement ;
- ▶ offrir une voie de signalement.

UN E-MAIL N'EST PAS UN COFFRE-FORT À MOTS DE PASSE

```
https://accounts.example/reset?token=<valeur-aleatoire-longue>
```

Le jeton

Généré par un CSPRNG, suffisamment long, stocké haché, lié au compte et à un but précis, avec expiration courte et usage unique.

La page

HTTPS seulement, politique de référent stricte, pas de ressources tierces inutiles, pas de jeton copié dans les journaux ou outils d'analyse.

Après succès

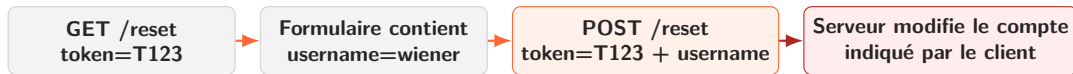
Invalider les autres jetons de récupération et proposer la révocation de toutes les sessions existantes.

Défaut	Conséquence	Correction
compte choisi dans le formulaire final	changement du mot de passe d'un autre utilisateur	compte dérivé du jeton côté serveur
jeton non vérifié au POST	contournement de la possession du lien	validation atomique à chaque étape
jeton réutilisable	plusieurs prises de contrôle	consommation unique
jeton prévisible	devinable ou énumérable	aléa cryptographique
sessions conservées	persistance d'un attaquant	révocation après changement

- 1 Demander une réinitialisation pour le compte de test autorisé.
- 2 Capturer la requête qui affiche puis celle qui valide le nouveau mot de passe.
- 3 Identifier où sont portés le jeton et l'identité du compte.
- 4 Vérifier si l'identité est dérivée du jeton ou acceptée depuis un paramètre client.
- 5 Confirmer avec le minimum d'actions et restaurer le compte de laboratoire.

Invariant attendu

Un jeton donné ne doit autoriser la modification que d'**un seul compte**, pour **une seule opération**, pendant **une courte durée**.



Vulnérabilité précise

Le jeton autorise l'opération, mais il n'est pas utilisé pour déterminer le propriétaire. Le paramètre `username` contrôlé par le client choisit le compte modifié.

- 1 Demander un lien pour le compte de test et l'ouvrir depuis la boîte du laboratoire.
- 2 Dans Proxy, distinguer la requête qui **affiche** le formulaire de celle qui **enregistre** le nouveau mot de passe.
- 3 Envoyer le POST final dans Repeater et repérer token, username, new-password-1 et new-password-2.
- 4 Conserver le jeton valide, mais remplacer seulement l'identifiant par la cible prévue par le laboratoire.
- 5 Soumettre, puis vérifier si le nouveau secret ouvre le compte cible.

Preuve de faille

Un jeton émis pour A modifie B : la liaison jeton-compte est absente.

Correction

Le serveur retrouve le compte depuis le hachage du jeton stocké, jamais depuis un champ client.

Empoisonnement du lien via l'en-tête Host

```
POST /forgot-password HTTP/1.1
Host: attacker-controlled.example

username=victim
```

Scénario

Si l'application construit le lien absolu à partir d'un en-tête non fiable, l'e-mail peut pointer vers un domaine tiers tout en contenant le vrai jeton.

Impact

La victime clique ; le jeton est transmis au domaine contrôlé, puis peut être utilisé sur le véritable site.

Correction

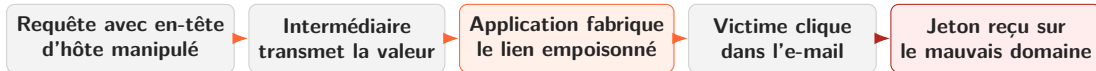
Utiliser une origine canonique configurée côté serveur et une liste stricte d'hôtes autorisés.

- 1 Cartographier le proxy inverse, le serveur applicatif et les en-têtes de réécriture utilisés dans le laboratoire.
- 2 Modifier un seul en-tête d'hôte à la fois et demander une réinitialisation du compte de test.
- 3 Examiner le lien reçu dans la boîte e-mail du laboratoire.
- 4 Vérifier si le domaine, le schéma ou le port provient d'une entrée contrôlable.
- 5 Démontrer l'exposition du jeton uniquement avec le serveur d'exploitation fourni.

Leçon d'architecture

« Interne » ne signifie pas « fiable ». Chaque intermédiaire doit supprimer, réécrire ou valider explicitement les en-têtes qui influencent les URL sensibles.

TP empoisonnement : chaîne complète



Vulnérabilité précise

Une donnée de routage provenant de la requête devient une donnée de sécurité utilisée pour construire le lien de récupération.

Point d'observation du TP

Le serveur d'exploitation du laboratoire reçoit une requête contenant le jeton dans l'URL ; cela prouve sa divulgation sans accéder à un système réel.

Entrée testée	Observation	Conclusion
Host	refusé ou normalisé	contrôle direct présent
X-Forwarded-Host	domaine repris dans l'e-mail	intermédiaire ou application lui fait confiance
X-Forwarded-Proto paramètre de requête	schéma du lien modifié aucun effet	construction d'URL influençable entrée non utilisée ici

Correction d'architecture

Construire les liens depuis une origine canonique configurée, rejeter les hôtes inconnus et faire supprimer les en-têtes entrants avant que le proxy n'ajoute ses propres valeurs fiables.

Parcours sûr

- ▶ session pleinement authentifiée ;
- ▶ réauthentification avec le secret actuel ;
- ▶ MFA pour le risque élevé ;
- ▶ compte dérivé de la session ;
- ▶ validation du nouveau mot de passe ;
- ▶ notification et révocation des sessions.

Failles possibles

Identité fournie dans un champ modifiable, messages différents, absence de limitation, vérification du secret actuel facultative ou endpoint accessible sans session complète.

ACTION SENSIBLE = RÉAUTHENTIFICATION

- 1 Capturer la requête de changement sur le compte de laboratoire.
- 2 Repérer l'ancien mot de passe, le nouveau et l'identité éventuelle.
- 3 Tester les combinaisons d'erreurs : ancien faux, nouveaux différents, session absente.
- 4 Observer si une branche révèle que l'ancien mot de passe est correct.
- 5 Vérifier la limitation propre à cette route.

Erreur classique

Le formulaire de changement devient un second endpoint de connexion, parfois sans les protections appliquées à /login.

Correction

Réponse uniforme, limitation commune et logique d'authentification centralisée.

TP changement de mot de passe : oracle logique

Ancien secret	Nouveaux secrets	Réponse
faux	identiques	« Current password is incorrect »
faux	différents	« New passwords do not match »
correct	différents	« New passwords do not match »
correct	identiques	mot de passe modifié

Vulnérabilité précise

L'ordre des validations révèle que l'ancien mot de passe est correct : lorsque les nouveaux champs diffèrent, le message devient un oracle utilisable pour tester l'ancien secret.

Condition aggravante

La route de changement n'applique pas nécessairement la même limitation que la page de connexion.

- 1 Conserver les deux nouveaux mots de passe volontairement différents.
- 2 Dans Intruder, marquer uniquement le champ de l'ancien mot de passe.
- 3 Utiliser la liste réduite prévue par le laboratoire.
- 4 Rechercher la réponse dont le message change vers « nouveaux mots de passe différents ».
- 5 Confirmer ce candidat via la connexion normale du compte de test.

Pourquoi l'astuce fonctionne

La différence entre les nouveaux champs empêche le changement réel, mais laisse le serveur révéler si l'ancien secret a franchi sa vérification.

Correction

Limitation centralisée, réauthentification dédiée et réponse externe uniforme quel que soit l'ordre des validations.

SYNTHÈSE

Auditer et sécuriser l'authentification

- 1 **Cartographier** tous les parcours : connexion, MFA, API, mobile, récupération, changement et persistance.
- 2 **Établir les états** : anonyme, pré-authentifié, MFA validée, session complète, verrouillé.
- 3 **Comparer les réponses** : contenu, statut, longueur, temps, cookies et redirections.
- 4 **Tester les transitions** : accès direct, paramètres modifiés, réutilisation, expiration, remise à zéro.
- 5 **Évaluer l'automatisation** : limitation, verrouillage, détection et contournements logiques.
- 6 **Confirmer puis corriger** l'invariant violé avec une preuve minimale et reproductible.

Identifiants

- ▶ phrases longues et mots compromis bloqués ;
- ▶ hachage lent, salé et paramétré ;
- ▶ messages et temps uniformes ;
- ▶ limitation adaptative ;
- ▶ MFA recommandée ou imposée selon le risque.

Sessions

- ▶ identifiants aléatoires ;
- ▶ régénération après authentification ;
- ▶ cookies sécurisés ;
- ▶ expiration et révocation ;
- ▶ invalidation après récupération ou changement sensible.

MFA

- ▶ état pré-authentifié côté serveur ;
- ▶ défi lié au compte et à la session ;
- ▶ expiration, usage unique, peu d'essais ;
- ▶ aucune route de contournement ;
- ▶ méthodes résistantes au phishing privilégiées.

Récupération

- ▶ réponse générique ;
- ▶ jeton aléatoire stocké haché ;
- ▶ origine canonique configurée ;
- ▶ compte dérivé du jeton ;
- ▶ notification et révocation après succès.

Événement		Signal utile	Réaction possible
nombreux comptes,	même secret	pulvérisation de mots de passe	ralentir, alerter, exiger MFA
nombreux secrets,	même compte	force brute ciblée	délai progressif, notification
défis MFA répétés		brute force ou fatigue push	invalider, réauthentifier, alerter
récupération inhabituelle		prise de contrôle potentielle	contrôle renforcé, révocation
nouvel appareil / pays		contexte anormal	step-up et avis utilisateur

Ne jamais journaliser

Mots de passe, codes MFA, jetons de session ou de réinitialisation complets.

- ❶ L'authentification prouve l'identité ; l'autorisation décide des actions permises.
- ❷ Une différence de message, de taille, d'état ou de temps peut devenir un oracle.
- ❸ Une limitation fragile ralentit parfois l'attaque sans réellement l'empêcher.
- ❹ La MFA échoue si son état n'est pas imposé côté serveur et lié au bon compte.
- ❺ Les mécanismes annexes doivent être au moins aussi sûrs que la connexion principale.

IDENTITÉ

PREUVE

ÉTAT SERVEUR

DÉFENSE EN PROFONDEUR

Questions ?

Comprendre la logique pour mieux protéger les identités

Tests uniquement dans un environnement autorisé

BURP SUITE ACADEMY